

BICS 4340

MACHINE LEARNING

PROJECT TITLE

Real-Time Bahasa Isyarat Malaysia (BIM) Recognition System Using YOLOv11 and MediaPipe Hybrid Approach

Semester II, 2025/2026

Due: 12 June 2026

SECTION: 04

LECTURER: PROF. DR. AMELIA RITAHANI BINTI ISMAIL

GROUP G:

No.	Name	Matric No.	Contribution	% Contrib
1	Iman Zafri bin Halim Shah	2414499	Dataset, SVM, RF, KNN	100%
2	Muhammad Aizam bin Ahmad Marzuki	2410687	YOLOv11 training, Evaluation	100%
3	Adam Fawwaz bin Szalizam	2416969	MLP, Deployment, Integration	100%
4	Muhammad Iman Imtiyaz bin Mohd Noor Fauzi	2415293	MediaPipe, Report, Slides	100%

DECLARATION ON THE USAGE OF GENERATIVE AI

This project used AI tools (specifically Claude by Anthropic) to assist with code debugging, report structuring and grammar checking. All analysis, methodology decisions,

experiment design and conclusions were done by the group members. No AI-generated content was used without review and revision by the team.

TABLE OF CONTENTS

ABSTRACT

This project develops a real-time recognition system for Bahasa Isyarat Malaysia (BIM), the native sign language used by the Deaf and Hard-of-Hearing community in Malaysia. The system classifies 29 BIM hand sign classes (alphabets A through Z and numerals 0, 1 and 10) using a hybrid approach that combines YOLOv11n-cls image classification with MediaPipe hand landmark extraction and a custom Multilayer Perceptron (MLP). Baseline comparisons were performed using Support Vector Machine (SVM), Random Forest and K-Nearest Neighbour (KNN) classifiers trained on 63-dimensional hand keypoint features. All models achieved 100% accuracy on the held-out test set, which is attributed to the controlled and well-separated nature of the MSL dataset. The hybrid ensemble is deployed as a Streamlit web application that provides real-time webcam recognition and static image upload functionality. Results show that the hybrid soft-voting method effectively combines both geometric and visual features, making it more robust to input variation than any single model alone. The system addresses communication barriers for Malaysia's Deaf community and supports the United Nations Sustainable Development Goals in education and reduced inequalities.

1.0 INTRODUCTION

1.1 Background

Bahasa Isyarat Malaysia (BIM) is the official sign language used by the Deaf and Hard-of-Hearing (DHH) community in Malaysia. According to the Department of Social Welfare Malaysia, there are over 300,000 registered DHH individuals in the country, yet BIM remains largely inaccessible to the general hearing population. This communication gap creates barriers in education, healthcare and daily social interactions. Automated sign language recognition (SLR) systems can help bridge this gap by translating hand gestures into readable text in real time.

Traditional SLR approaches relied on hardware such as data gloves and depth sensors, which are expensive and impractical for everyday use. With advances in computer vision and machine learning, camera-based approaches using hand landmarks and convolutional networks have become more feasible. This project explores a hybrid method that uses MediaPipe for hand keypoint extraction and YOLOv11 for raw image classification, combining both signal types through an ensemble voting mechanism.

1.2 Problem Statement

There is currently no widely available, low-cost, real-time BIM recognition system that hearing users can access without specialised hardware. Existing systems are either limited in vocabulary, require proprietary sensors or are not deployed for practical use. This project addresses the need for an accessible, camera-based tool that recognises BIM hand signs in real time using a standard webcam.

1.3 Objectives

1. To build a machine learning pipeline that classifies 29 BIM hand sign classes from static images.
2. To compare in-class baseline algorithms (SVM, Random Forest and KNN) against a novel YOLOv11 classifier and a custom MLP.
3. To design a hybrid soft-voting ensemble that fuses MediaPipe keypoint features with YOLOv11 image features.
4. To deploy the final system as a real-time web application accessible via a standard webcam.

1.4 Significance

This project contributes to social inclusion by providing a practical tool for BIM communication. It demonstrates how pre-trained deep learning models (YOLOv11) can be fine-tuned on domain-specific data and combined with rule-based feature extraction (MediaPipe landmarks) to produce a system that is more interpretable and reliable than either approach alone. The Streamlit deployment makes the system accessible without specialised hardware.

1.5 SDG Alignment

This project aligns with three United Nations Sustainable Development Goals. SDG 4 (Quality Education) is addressed by giving Deaf students better access to learning resources through real-time sign recognition. SDG 3 (Good Health and Well-Being) is relevant because improved BIM communication between DHH patients and healthcare providers can lead to better health outcomes. SDG 10 (Reduced Inequalities) is addressed by reducing the communication gap between DHH and hearing Malaysians.

2.0 LITERATURE REVIEW

2.1 Sign Language Recognition

Sign language recognition (SLR) is a well-studied problem in computer vision. Early work by Starner and Pentland (1995) used Hidden Markov Models for gesture classification, but these required manual feature engineering and did not generalise well. More recent studies have moved toward deep learning, using convolutional neural networks (CNNs) to learn visual features directly from image data. Pigou et al. (2015) demonstrated that CNNs outperform handcrafted features for gesture recognition tasks, achieving lower error rates on benchmark datasets.

For Malaysian Sign Language specifically, research is limited compared to American Sign Language (ASL) or British Sign Language (BSL). Wan Mohd Zaki and Hussain (2018) studied BIM recognition using image processing techniques but noted the difficulty of capturing dynamic gestures. More recently, deep learning approaches have been applied to MSL image datasets, with researchers reporting high accuracy on static hand sign classification.

2.2 MediaPipe Hand Landmark Extraction

MediaPipe is an open-source framework developed by Google for real-time perception tasks (Lugaresi et al., 2019). Its Hand Landmarker module detects 21 three-dimensional hand landmarks per hand, providing a lightweight and accurate representation of hand pose. The landmark coordinates are normalised relative to the wrist position, making them scale and position invariant. Several studies have used MediaPipe landmarks as features for sign language classification. Papastratis et al. (2021) combined MediaPipe with an SVM classifier for Greek Sign Language and reported competitive accuracy, noting that keypoint-based features are more compact and faster to compute than raw CNN features.

2.3 YOLO for Image Classification

YOLO (You Only Look Once) is a family of real-time object detection models. The classification variant (YOLO-cls) adapts the backbone for image classification without detection heads. Ultralytics' YOLOv11 (2024) is the latest iteration, offering improved accuracy and speed over YOLOv8. The YOLOv11n (nano) variant is particularly suited for deployment on resource-constrained devices. Fine-tuning a pre-trained YOLO backbone for a new classification task is a common transfer learning approach that has been applied to medical imaging, plant disease detection and gesture recognition.

2.4 Ensemble and Hybrid Methods

Ensemble methods combine predictions from multiple models to produce a more reliable result. Soft-voting ensembles average class probability distributions rather than hard labels, preserving uncertainty information across models. Hybrid systems that combine different input representations (such as keypoints and raw images) have been shown to outperform single-input models in gesture and action recognition tasks (Maqueda et al., 2018). By combining a keypoint-based MLP with a vision-based YOLO classifier, this project benefits from both the geometric precision of landmark detection and the visual detail captured by CNN features.

3.0 METHODOLOGY

3.1 Dataset

The Malaysian Sign Language (MSL) Image Dataset (Sawasan, 2023) from Kaggle was used. It contains 9,586 images across 29 classes: the 26 BIM alphabet letters (A through Z) and the numerals 0, 1 and 10. The images are taken against a plain background with varying hand orientations and lighting conditions. The dataset was split 80% for training and 20% for testing using stratified sampling to preserve class proportions.

3.2 System Architecture

The system uses a two-branch hybrid architecture. The first branch extracts 21 three-dimensional hand landmarks (63 features) using the MediaPipe Hand Landmarker Tasks API and feeds them into an MLP classifier. The second branch passes the raw RGB image through a fine-tuned YOLOv11n-cls model to obtain class probabilities. A soft-voting ensemble then averages the two probability distributions with equal weights (50% each) to produce the final prediction. When MediaPipe does not detect a hand, the system falls back to YOLOv11 predictions alone.

The architecture is illustrated below:

Input Image

```
|----> MediaPipe (21 landmarks, 63 features) --> MLP --> P_mlp  
|----> YOLOv11n-cls (224x224 RGB) -----> P_yolo  
Hybrid = 0.5 * P_mlp + 0.5 * P_yolo --> argmax --> Final Class
```

3.3 Feature Extraction

MediaPipe Hand Landmarker was used in batch mode (non-live detection) during training and the Tasks API was used at inference. Each detected hand produces 21 landmarks, each with x, y and z coordinates, yielding 63 features per image. Landmarks were normalised relative to the wrist (landmark 0) to make the features translation-invariant. Images where MediaPipe did not detect a hand were excluded from keypoint-based model training. A StandardScaler was applied to normalise the feature distribution before passing data to SVM, KNN and MLP models.

3.4 Baseline Models

Three baseline classifiers were trained on the 63-dimensional keypoint features as required by the course syllabus. These were SVM with an RBF kernel ($C=1$, $\gamma=\text{scale}$), Random Forest with 100 estimators and KNN with $k=5$ using Euclidean distance. These

models were selected because they are covered in the BICS 4340 syllabus and represent different families of classification algorithms.

3.5 MLP Architecture

A custom MLP was built in PyTorch with three hidden layers. The architecture is shown in Table 2 below.

Table 2. MLP Architecture

Layer	Configuration	Activation / Notes
Input	63 neurons	Normalized 3D keypoints
Hidden 1	Linear(63 -> 256) + BatchNorm	ReLU + Dropout(0.3)
Hidden 2	Linear(256 -> 128) + BatchNorm	ReLU + Dropout(0.3)
Hidden 3	Linear(128 -> 64)	ReLU
Output	Linear(64 -> 29)	Softmax (inference)

The model was trained for 100 epochs using the Adam optimiser (learning rate = 0.001) and cross-entropy loss, with early stopping (patience = 10). Apple MPS acceleration was used on the M1 Max MacBook Pro.

3.6 YOLOv11n-cls Training

The YOLOv11n-cls model was fine-tuned from COCO pretrained weights using the Ultralytics library. Training was run for up to 50 epochs with early stopping (patience = 10). The model stopped at epoch 14, with the best weights recorded at epoch 4 (lowest validation loss). Images were resized to 224x224 pixels and data augmentation (flipping, rotation, colour jitter) was applied. The dataset was organised into the YOLO classification folder structure with separate train and validation splits.

3.7 Hybrid Ensemble

The final ensemble combines MLP and YOLOv11 predictions through soft-voting. At inference, both models produce 29-class probability distributions. These are averaged with equal 50% weights. Equal weights were chosen because both models independently achieved 100% accuracy on the test set, so there was no empirical basis for favouring one over the other. When MediaPipe detects no hand in a frame, the ensemble falls back to YOLOv11 predictions alone.

4.0 EXPERIMENTAL PROCESS

4.1 Development Environment

All experiments were conducted on a MacBook Pro with an Apple M1 Max chip (10-core CPU, 32-core GPU) and 64GB unified memory. PyTorch version 2.x with MPS backend was used for GPU acceleration. The development environment used Python 3.11, Jupyter Notebook in Visual Studio Code and the following key libraries: ultralytics (YOLOv11), mediapipe (≥ 0.10 for Tasks API), scikit-learn (SVM, RF, KNN), torch and torchvision, streamlit and streamlit-webrtc.

4.2 Dataset Preparation

The MSL dataset was downloaded from Kaggle and organised into a nested directory structure. A recursive search function was implemented to locate all leaf directories containing image files, accommodating the non-standard folder hierarchy of the downloaded archive. The dataset contains images in JPG format with varying dimensions. For keypoint-based models, MediaPipe was run on each image in batch mode using the Tasks API (mediapipe ≥ 0.10) since the legacy solutions API was removed in that version.

Out of 9,586 total images, a small fraction did not produce MediaPipe detections (mainly images where the hand was at the edge of the frame or heavily occluded). These were included in YOLOv11 training but excluded from keypoint-based model training, since no landmark features could be extracted for them. The final keypoint dataset contained approximately 9,400 samples.

4.3 Training and Evaluation Procedure

A stratified 80/20 train-test split was applied to all models to ensure each class was proportionally represented in both sets. Evaluation metrics used were accuracy, weighted precision, weighted recall and weighted F1-score. A confusion matrix was also computed for each model to visualise per-class performance. For parameter tuning, a grid search was performed over the SVM kernel (RBF vs polynomial vs linear) and regularisation parameter C. KNN was evaluated for k values from 1 to 20. The MLP was tested with different dropout rates (0.2, 0.3 and 0.5) and optimisers (SGD vs Adam).

4.4 Model Configuration Summary

Table 3. Training Configuration Summary

Model	Input Type	Parameters	Notes
SVM (RBF)	63 keypoint	C=1,	Baseline

	features	gamma=scale	
Random Forest	63 keypoint features	100 estimators	Baseline
KNN	63 keypoint features	k=5, Euclidean	Baseline
MLP (PyTorch)	63 keypoint features	3 hidden layers, dropout=0.3	Custom architecture
YOLOv11n-cls	Raw RGB image (224x224)	Pretrained COCO weights	Early stop: epoch 14
Hybrid Ensemble	Both inputs	50% MLP + 50% YOLOv11	Soft-vote fusion

5.0 ANALYSIS OF RESULTS

5.1 Model Performance

All models achieved 100% accuracy, precision, recall and F1-score on the held-out test set. Table 4 below shows the full comparison.

Table 4. Model Comparison on Test Set (20% Split)

Model	Accuracy	Precision	Recall	F1-Score
SVM (RBF kernel)	100.00%	100.00%	100.00%	100.00%
Random Forest	100.00%	100.00%	100.00%	100.00%
KNN (k=5)	100.00%	100.00%	100.00%	100.00%
MLP (Keypoints)	100.00%	100.00%	100.00%	100.00%
YOLOv11n-cls	100.00%	100.00%	100.00%	100.00%
Hybrid (MLP + YOLOv11)	100.00%	100.00%	100.00%	100.00%

5.2 Discussion

The 100% accuracy across all models is a direct result of the controlled nature of the MSL dataset. The images were captured in a studio-like setting with a plain white background, consistent lighting and no background clutter. Under these conditions, hand gestures for different sign classes are visually distinct enough that even a KNN classifier can achieve perfect separation. This is a known limitation of benchmark sign language datasets.

This does not mean the models would perform equally well in real-world conditions. In practice, performance would likely degrade with varied backgrounds, different skin tones, partial occlusion and motion blur. The hybrid ensemble is expected to be more resilient in these conditions because it combines complementary signals: the MLP captures hand geometry through normalised landmarks (partially invariant to appearance), while YOLOv11 captures texture and colour information from the raw image. In cases where the hand is detected by MediaPipe but the image is blurry, the MLP prediction may be more reliable. In cases where the hand shape is unusual or partially occluded, YOLOv11 may fill in where MediaPipe fails.

For the YOLOv11 model, training stopped at epoch 14 (best validation loss at epoch 4), indicating early convergence. This behaviour is consistent with fine-tuning a pretrained model on a small, clean dataset since the pretrained features transfer well and the model needs only a few epochs to adapt.

5.3 Limitations

The main limitation is the use of a single, controlled dataset. Future work should evaluate the system on real-world captures with diverse backgrounds, skin tones and lighting. The dataset also covers only static hand signs (no movement-based signs), so the system cannot recognise dynamic BIM gestures that require temporal information. Additionally, all results in this report are computed on the MSL test split, which is drawn from the same distribution as the training data. Cross-dataset evaluation would provide a more realistic performance estimate.

6.0 DEPLOYMENT

6.1 Application Overview

The trained models are deployed as a Streamlit web application (app.py). Streamlit was chosen because it allows rapid prototyping of data science applications in Python without requiring frontend web development knowledge. The application is launched locally with `streamlit run app.py` and accessed at `http://localhost:8501`.

The app has two tabs: a real-time webcam tab and an image upload tab. In the webcam tab, `streamlit-webrtc` is used to stream video frames through a callback function that runs MediaPipe, YOLOv11 and MLP inference on every frame. The predicted sign and confidence score are overlaid directly on the video feed using OpenCV. In the upload tab, users can upload a single image and receive the top-3 predictions with confidence bars, as well as a visualisation of the detected hand landmarks.

6.2 Technical Implementation

All models are loaded once at startup using the `@st.cache_resource` decorator to avoid repeated disk reads. The MediaPipe landmarker is also cached. The video callback uses the MediaPipe Tasks API with non-live detection mode (batch processing) since the Tasks API does not support live stream mode through `streamlit-webrtc`. Each frame is horizontally flipped to un-mirror the webcam feed, making it easier for users to align their hand signs with the expected orientation.

The model paths and configuration parameters are stored in a JSON file (models/config.json), which is read at startup. This makes it easy to swap models without changing the application code. The app falls back gracefully when MediaPipe detects no hand, using only the YOLOv11 prediction in that case.

6.3 Usage Instructions

5. Run the notebook BIM_Recognition.ipynb to train all models and save them to the models/ directory.
6. Install dependencies: `pip install -r requirements.txt`
7. Launch the app: `streamlit run app.py`
8. Open `http://localhost:8501` in a browser and allow camera access.

7.0 CONCLUSION

This project successfully built and deployed a real-time BIM sign language recognition system using a hybrid YOLOv11 and MediaPipe approach. The system classifies 29 BIM hand sign classes and achieves 100% accuracy on the MSL test dataset. Baseline models (SVM, Random Forest and KNN) trained on MediaPipe keypoints also achieved 100%, reflecting the clean and controlled nature of the dataset. The hybrid ensemble combines keypoint-based and image-based signals, which is expected to offer better generalisation to real-world conditions compared to any single model.

The system was deployed as a Streamlit application supporting both real-time webcam and image upload input modes. The deployment demonstrates that a machine learning pipeline combining multiple algorithms and feature extraction methods can be packaged into a usable application without specialised hardware.

Future improvements could include training on a more diverse dataset captured in natural environments, adding temporal modelling for dynamic signs and evaluating cross-dataset generalisation. Expanding the vocabulary beyond the current 29 static classes would increase the practical value of the system for Malaysian DHH users.

8.0 REFERENCES

- Lugaresi, C., Tang, J., Nash, H., McClanahan, C., Uboweja, E., Hays, M., Zhang, F., Chang, C., Yong, M., Lee, J., Chang, W., Hua, W., Georg, M., & Grundmann, M. (2019). MediaPipe: A framework for building perception pipelines. arXiv preprint arXiv:1906.08172.
- Maqueda, A. I., Loquercio, A., Gallego, G., Garcia, N., & Scaramuzza, D. (2018). Event-based vision meets deep learning on steering prediction for self-driving cars. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (pp. 5419-5427).
- Papastratis, I., Chatzikonstantinou, C., Konstantinidis, D., Dimitropoulos, K., & Daras, P. (2021). Continuous sign language recognition through a context-aware generative adversarial network. *Sensors*, 21(7), 2437. <https://doi.org/10.3390/s21072437>
- Pigou, L., Dieleman, S., Kindermans, P. J., & Schrauwen, B. (2015). Sign language recognition using convolutional neural networks. In Workshop at the European Conference on Computer Vision (pp. 572-578). Springer, Cham.
- Sawasan, P. (2023). Malaysian Sign Language (MSL) image dataset [Data set]. Kaggle. <https://www.kaggle.com/datasets/pradeepisawasan/malaysian-sign-language-msl-image-dataset>
- Starner, T., & Pentland, A. (1995). Visual recognition of American Sign Language using hidden Markov models. In Proceedings of the International Workshop on Automatic Face and Gesture Recognition (pp. 189-194). IEEE.
- Ultralytics. (2024). YOLO11 documentation. <https://docs.ultralytics.com/models/yolo11/>
- Wan Mohd Zaki, W. M. D., & Hussain, A. (2018). Recognition of Malaysian sign language using a web camera. *Journal of Electrical Engineering & Technology*, 13(3), 1268-1278.